

NutriCoach Scalable Architecture

1M-User AI Nutrition SaaS

Table of Contents

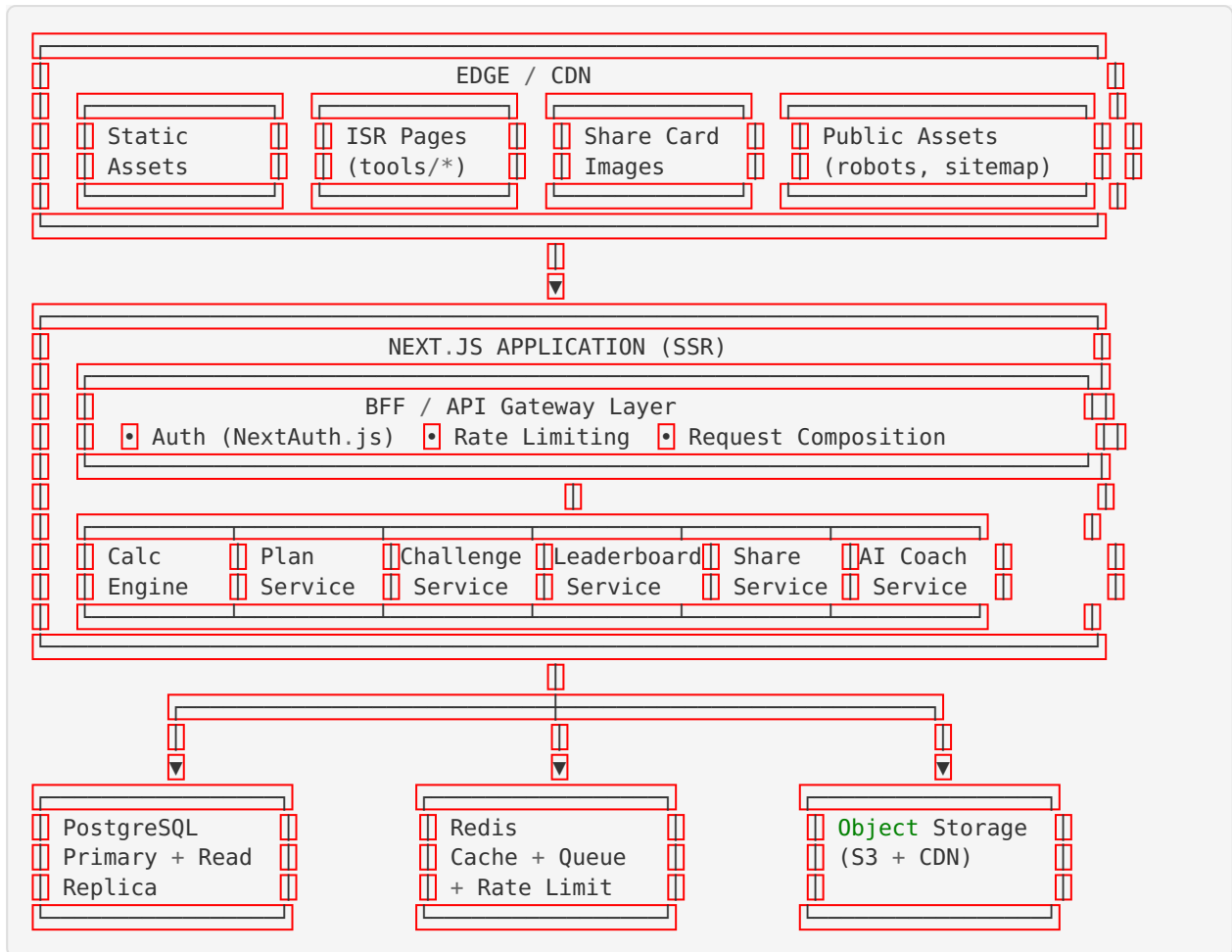
1. [Architecture Overview](#)
 2. [System Diagram](#)
 3. [Service Boundaries](#)
 4. [Data Layer](#)
 5. [Performance Patterns](#)
 6. [Observability](#)
 7. [Deployment Plan](#)
 8. [Scaling Plan](#)
-

Architecture Overview

NutriCoach follows a **modular monolith** architecture that can evolve into microservices as scale demands. The system is designed around these principles:

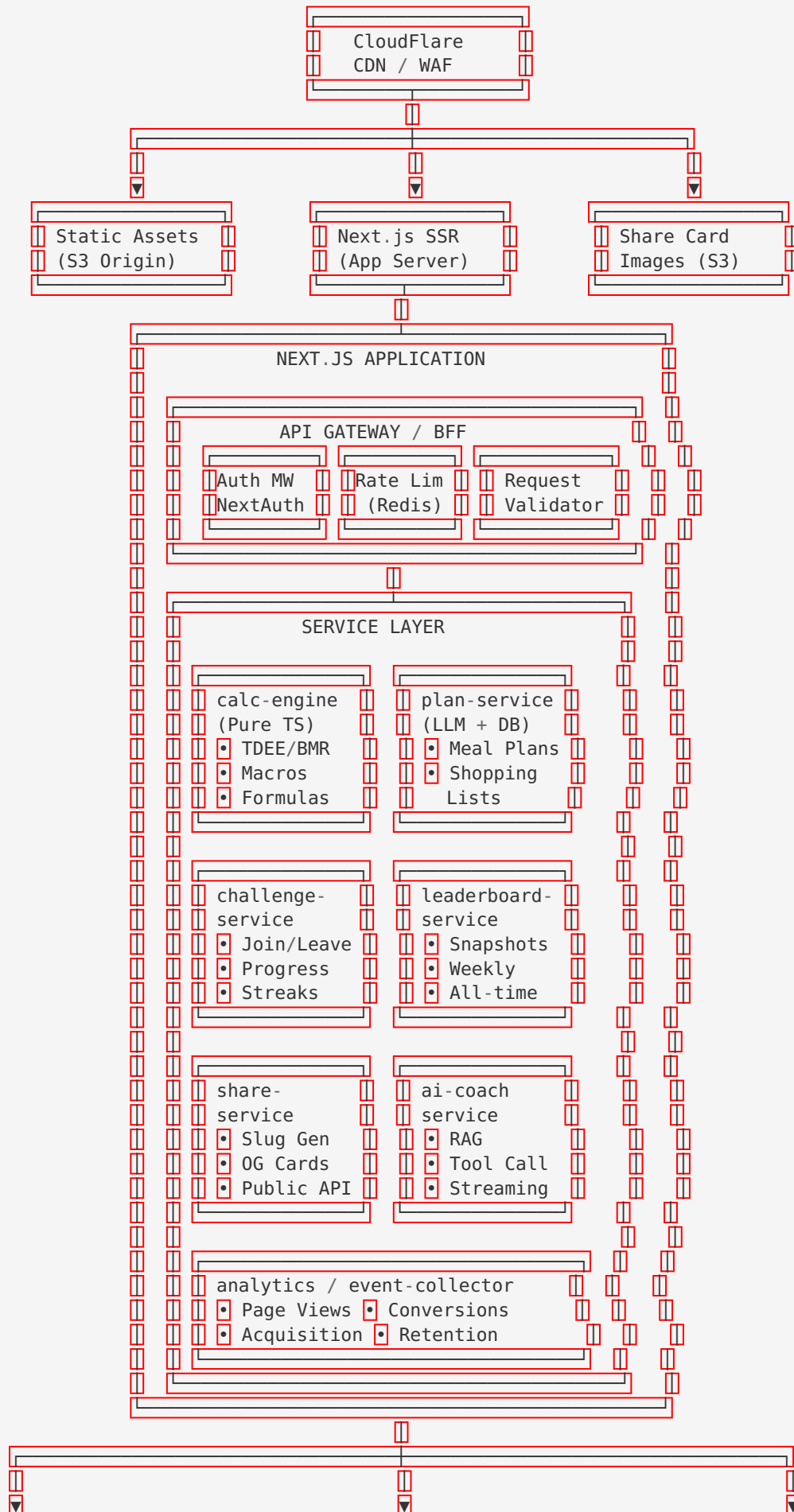
- **SSR/ISR** for SEO-critical pages (calculators, public shares, leaderboards)
- **BFF Pattern** for API composition and security
- **Event-Driven** for async heavy operations
- **CQRS-lite** for read-heavy leaderboards via snapshot aggregation
- **Edge Caching** for public content

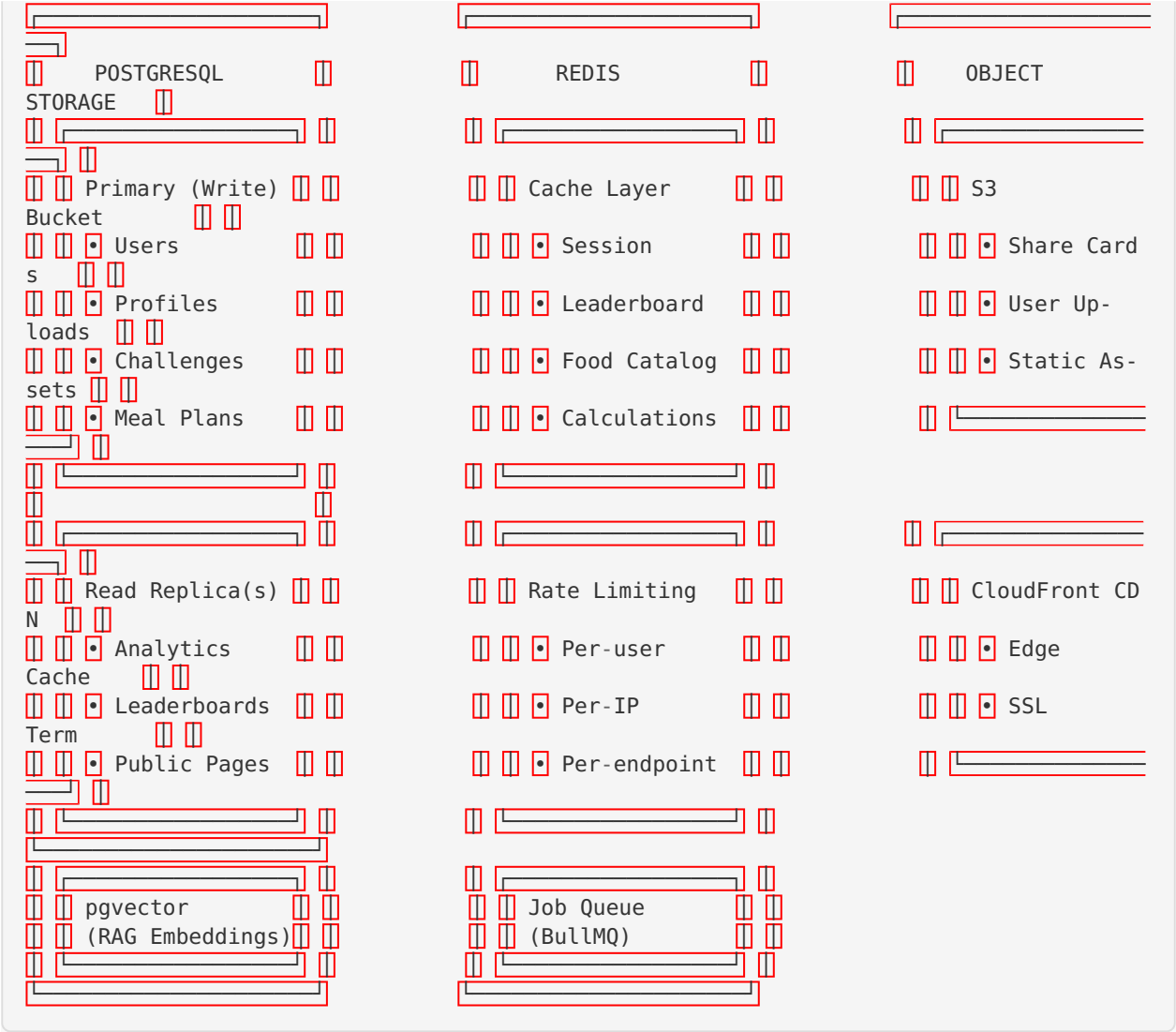
High-Level Flow



System Diagram

Detailed Component Architecture





Service Boundaries

1. BFF / API Gateway

Responsibilities:

- Authentication & authorization
- Rate limiting (Redis-backed)
- Request validation
- Response composition
- Error handling & logging

Endpoints:

POST	/api/auth/*	# NextAuth.js handlers
GET	/api/health	# Health check
ALL	/api/*	# Protected routes (require auth)

2. Calc Engine Service

Location: `lib/calc-engine.ts` (Pure TypeScript, no I/O)

Responsibilities:

- Deterministic nutrition calculations
- Formula versioning
- Input validation

Key Functions:

```
// Pure functions - no database, no side effects
calculateNutrition(input: ProfileInput): NutritionResult
adjustCaloriesForGoal(base: number, goal: Goal): number
calculateMacroGrams(calories: number, dist: MacroDistribution): MacroGrams
validateProfileInput(input: unknown): ValidationResult
```

Design Principle: The AI Coach service CANNOT modify these calculations. All nutrition math is deterministic and versioned.

3. Plan Service

Location: `lib/plan-service.ts`

Responsibilities:

- Weekly meal plan generation (LLM-powered)
- Shopping list aggregation
- Plan caching

Endpoints:

```
POST    /api/meal-plan/generate    # Generate new plan (async job)
GET     /api/meal-plan/:id        # Fetch existing plan
GET     /api/meal-plan/shopping  # Get shopping list for plan
DELETE  /api/meal-plan/:id       # Delete plan
```

Async Flow:

```
User Request → Queue Job → Worker Generates → Webhook/Poll → User Notified
```

4. Challenge/Streak Service

Location: `lib/challenge-service.ts` , `lib/streak-system.ts`

Responsibilities:

- Challenge lifecycle management
- Progress tracking
- Streak calculations
- Achievement unlocks

Endpoints:

GET	/api/challenges	# List available challenges
POST	/api/challenges/:id/join	# Join a challenge
GET	/api/challenges/:id/progress	# Get user progress
POST	/api/challenges/:id/check-in	# Daily check-in
GET	/api/streak	# Get current streak
POST	/api/streak/activity	# Record activity

5. Leaderboard Service

Location: lib/leaderboard-service.ts

Responsibilities:

- Snapshot-based ranking computation
- Weekly and all-time leaderboards
- Efficient pagination

Endpoints:

GET	/api/leaderboard/weekly	# Weekly top users
GET	/api/leaderboard/all-time	# All-time top users
GET	/api/leaderboard/challenge/:id	# Challenge-specific
GET	/api/leaderboard/user/:id/rank	# User's current rank

Snapshot Strategy:

- Leaderboard NOT computed from raw events in real-time
- Hourly cron job aggregates `LeaderboardSnapshot` table
- API reads from snapshot cache (Redis) or snapshot table

6. Share Service

Location: lib/share-service.ts

Responsibilities:

- Slug generation (NanoID)
- Public payload construction
- OG share card image generation
- Public page data serving

Endpoints:

POST	/api/share/challenge/:id	# Create share link
GET	/api/share/:slug	# Get public payload (SSR)
GET	/api/share/:slug/card	# Get OG image URL
POST	/api/share/generate-card	# Async card generation

Public SSR Routes:

```

/s/:slug          # User share pages
/c/:slug          # Challenge share pages
/leaderboard      # Public leaderboard

```

7. AI Coach Service

Location: `lib/ai-coach.ts` , `lib/rag-system.ts`

Responsibilities:

- RAG-based context retrieval
- Tool calling (read-only access to calc-engine)
- Response streaming
- Conversation history

Endpoints:

```

POST   /api/chat                # Send message (streaming)
GET    /api/chat/sessions     # List sessions
GET    /api/chat/sessions/:id/messages # Get session history

```

Critical Constraint:

The AI Coach can READ calculation results but CANNOT MODIFY deterministic nutrition formulas. It can only explain, suggest, and guide.

8. Analytics / Event Collector

Location: `lib/analytics.ts` , `lib/acquisition.ts`

Responsibilities:

- Event ingestion
- Acquisition source tracking
- Conversion funnel analysis
- Growth metrics

Endpoints:

```

POST   /api/analytics/event    # Track event (batched)
POST   /api/acquisition/track  # Track acquisition
GET    /api/analytics/acquisition # Dashboard data
GET    /api/analytics/retention  # Retention curves

```

Event Types:

- `PAGE_VIEW` , `CALCULATOR_USE` , `SIGNUP` , `PLAN_GENERATED`
- `CHALLENGE_JOINED` , `CHALLENGE_COMPLETED` , `REFERRAL_CONVERTED`

Data Layer

PostgreSQL Schema Extensions

```
// =====
// LEADERBOARD SNAPSHOTS (CQRS Read Model)
// =====

model LeaderboardSnapshot {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)

  // Snapshot data
  periodType  String   // 'weekly' | 'all_time'
  periodKey   String   // '2026-W09' for weekly, 'all_time' for all-time

  // Aggregated metrics
  totalPoints Int       @default(0)
  streakDays  Int       @default(0)
  challengesCompleted Int @default(0)
  mealsLogged Int       @default(0)

  // Ranking (computed)
  rank        Int?

  // Timestamps
  computedAt  DateTime @default(now())

  @@unique([userId, periodType, periodKey])
  @@index([periodType, periodKey, totalPoints(sort: Desc)])
  @@index([computedAt])
}

// =====
// JOB QUEUE (for async processing)
// =====

model JobQueue {
  id          String   @id @default(cuid())
  jobType     String   // 'PLAN_GENERATION' | 'SHARE_CARD' | 'EMBEDDING' | 'SNAPSHOT'
  payload     Json
  status      String   @default("pending") // pending | processing | completed | failed
  priority    Int       @default(0)
  attempts    Int       @default(0)
  maxAttempts Int       @default(3)

  // Scheduling
  scheduledFor DateTime @default(now())
  startedAt    DateTime?
  completedAt  DateTime?

  // Results
  result       Json?
  error        String?

  // Timestamps
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  @@index([status, scheduledFor])
  @@index([jobType, status])
}
```

```
// =====
// EVENT STORE (Partitioned by Month)
// =====

model AnalyticsEvent {
  id          String   @id @default(cuid())
  userId      String?
  sessionId   String

  // Event data
  eventType   String
  eventData   Json?

  // Context
  source      String?  // seo | social | referral | direct | product_hunt
  referrer    String?
  userAgent   String?
  ipHash      String?  // Hashed for privacy

  // Partitioning key
  eventMonth  String   // '2026-03' - used for table partitioning

  // Timestamps
  createdAt   DateTime @default(now())

  @@index([eventMonth, eventType])
  @@index([userId, createdAt])
  @@index([sessionId])
}

// =====
// SHARE CARDS (S3 References)
// =====

model ShareCard {
  id          String   @id @default(cuid())
  shareId     String   @unique
  shareType   String   // 'challenge' | 'profile' | 'achievement'

  // S3 storage
  s3Key       String
  cdnUrl      String

  // Metadata
  width       Int      @default(1200)
  height      Int      @default(630)

  // Cache invalidation
  generatedAt DateTime @default(now())
  expiresAt   DateTime?

  @@index([shareId])
}

// =====
// RATE LIMIT TRACKING (backup to Redis)
// =====

model RateLimitLog {
  id          String   @id @default(cuid())
  identifier  String   // userId or IP
  endpoint    String
}
```

```

windowStart    DateTime
requestCount   Int          @default(1)

@@unique([identifier, endpoint, windowStart])
@@index([windowStart])

```



Redis Data Structures

```

# Session Cache
session:{sessionId}           ➡ JSON (user session)

# Rate Limiting
rl:{endpoint}:{userId}:{window} ➡ INT (request count)
rl:ip:{ip}:{window}           ➡ INT (request count)

# Leaderboard Cache
leaderboard:weekly:{periodKey} ➡ ZSET (userId ➡ score)
leaderboard:all_time           ➡ ZSET (userId ➡ score)
leaderboard:challenge:{id}     ➡ ZSET (userId ➡ score)

# Food Catalog Cache
foods:catalog                  ➡ JSON (full catalog)
foods:search:{query}           ➡ JSON (search results)

# Calculation Cache
calc:{userId}:latest           ➡ JSON (latest calculation)

# Job Queue (BullMQ)
bull:plan_generation:*         ➡ BullMQ structures
bull:share_card:*              ➡ BullMQ structures
bull:embedding:*               ➡ BullMQ structures
bull:snapshot:*                ➡ BullMQ structures

```

pgvector Setup (RAG)

```

-- Enable extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Embeddings table
CREATE TABLE embeddings (
  id SERIAL PRIMARY KEY,
  chunk_id VARCHAR(255) REFERENCES pdf_chunks(id),
  embedding vector(1536), -- OpenAI ada-002 dimensions
  created_at TIMESTAMP DEFAULT NOW()
);

-- HNSW index for fast similarity search
CREATE INDEX ON embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

```

Performance Patterns

1. Heavy Caching for Public Pages

```
// next.config.js ISR configuration
module.exports = {
  async headers() {
    return [
      {
        source: '/tools/:path*',
        headers: [
          { key: 'Cache-Control', value: 's-maxage=3600, stale-while-revalidate=86400' }
        ]
      },
      {
        source: '/c/:slug',
        headers: [
          { key: 'Cache-Control', value: 's-maxage=300, stale-while-revalidate=3600' }
        ]
      },
      {
        source: '/leaderboard',
        headers: [
          { key: 'Cache-Control', value: 's-maxage=60, stale-while-revalidate=300' }
        ]
      }
    ]
  }
}
```

2. Async Jobs for Heavy Tasks

```
// lib/queue.ts
import { Queue, Worker } from 'bullmq';

const connection = { host: process.env.REDIS_HOST, port: 6379 };

// Queues
export const planGenerationQueue = new Queue('plan_generation', { connection });
export const shareCardQueue = new Queue('share_card', { connection });
export const embeddingQueue = new Queue('embedding', { connection });
export const snapshotQueue = new Queue('snapshot', { connection });

// Example: Plan Generation Worker
const planWorker = new Worker('plan_generation', async (job) => {
  const { userId, calculationId } = job.data;
  // Generate meal plan via LLM
  // Save to database
  // Notify user via webhook/SSE
}, { connection, concurrency: 5 });
```

3. Leaderboard Snapshot Aggregation

```
// Hourly cron job
async function computeLeaderboardSnapshots() {
  const periodKey = getCurrentWeekKey(); // '2026-W09'

  // Aggregate from raw events
  const rankings = await prisma.$queryRaw`
    WITH user_scores AS (
      SELECT
        user_id,
        SUM(CASE WHEN event_type = 'MEAL_LOGGED' THEN 1 ELSE 0 END) as meals,
        SUM(CASE WHEN event_type = 'STREAK_MAINTAINED' THEN 1 ELSE 0 END) as
streak_days,
        SUM(CASE WHEN event_type = 'CHALLENGE_COMPLETED' THEN 10 ELSE 0 END) as chal-
lenge_pts,
        SUM(points) as total_points
      FROM analytics_events
      WHERE event_month >= ${getWeekStart(periodKey)}
      GROUP BY user_id
    )
    SELECT
      *,
      RANK() OVER (ORDER BY total_points DESC) as rank
    FROM user_scores
  `;

  // Upsert snapshots
  await prisma.leaderboardSnapshot.createMany({
    data: rankings.map(r => ({
      userId: r.user_id,
      periodType: 'weekly',
      periodKey,
      totalPoints: r.total_points,
      mealsLogged: r.meals,
      rank: r.rank,
      computedAt: new Date()
    })),
    skipDuplicates: true
  });

  // Update Redis cache
  await redis.del(`leaderboard:weekly:${periodKey}`);
  for (const r of rankings) {
    await redis.zadd(`leaderboard:weekly:${periodKey}`, r.total_points, r.user_id);
  }
}
```

4. Table Partitioning for Events

```
-- Create partitioned table
CREATE TABLE analytics_events (
  id UUID PRIMARY KEY,
  user_id UUID,
  event_type VARCHAR(50),
  event_data JSONB,
  event_month VARCHAR(7),
  created_at TIMESTAMP DEFAULT NOW()
) PARTITION BY LIST (event_month);

-- Create monthly partitions (automated via cron)
CREATE TABLE analytics_events_2026_03
PARTITION OF analytics_events
FOR VALUES IN ('2026-03');

CREATE TABLE analytics_events_2026_04
PARTITION OF analytics_events
FOR VALUES IN ('2026-04');

-- Drop old partitions after retention period
DROP TABLE IF EXISTS analytics_events_2025_03;
```

Observability

1. Structured Logging

```
// lib/logger.ts
import pino from 'pino';

export const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  formatters: {
    level: (label) => ({ level: label })
  },
  base: {
    service: 'nutrcoach',
    env: process.env.NODE_ENV,
    version: process.env.APP_VERSION
  }
});

// Usage
logger.info({ userId, action: 'meal_plan_generated', planId }, 'User generated meal plan');
logger.error({ err, userId, endpoint }, 'API request failed');
```

2. OpenTelemetry Tracing

```
// instrumentation.ts
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentations-node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({
    url: process.env.OTEL_EXPORTER_OTLP_ENDPOINT
  }),
  instrumentations: [
    getNodeAutoInstrumentations({
      '@opentelemetry/instrumentation-http': { enabled: true },
      '@opentelemetry/instrumentation-pg': { enabled: true },
      '@opentelemetry/instrumentation-redis': { enabled: true }
    })
  ]
});

sdk.start();
```


3. Metrics & Alerts

```
// lib/metrics.ts
import { Counter, Histogram, Gauge, Registry } from 'prom-client';

export const registry = new Registry();

// Request latency
export const httpRequestDuration = new Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests',
  labelNames: ['method', 'route', 'status'],
  buckets: [0.1, 0.5, 1, 2, 5]
});

// Queue depth
export const queueDepth = new Gauge({
  name: 'job_queue_depth',
  help: 'Number of jobs in queue',
  labelNames: ['queue_name', 'status']
});

// Cache hit rate
export const cacheHits = new Counter({
  name: 'cache_hits_total',
  help: 'Total cache hits',
  labelNames: ['cache_type']
});

export const cacheMisses = new Counter({
  name: 'cache_misses_total',
  help: 'Total cache misses',
  labelNames: ['cache_type']
});

// Error rate
export const errorCounter = new Counter({
  name: 'errors_total',
  help: 'Total errors',
  labelNames: ['type', 'endpoint']
});

registry.registerMetric(httpRequestDuration);
registry.registerMetric(queueDepth);
registry.registerMetric(cacheHits);
registry.registerMetric(cacheMisses);
registry.registerMetric(errorCounter);
```

Alert Rules (Prometheus/Grafana)

```
# alerts.yml
groups:
- name: nutricoach
  rules:
- alert: HighP95Latency
  expr: histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
  > 2
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "P95 latency above 2s"

- alert: HighQueueDepth
  expr: job_queue_depth{status="pending"} > 100
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "Job queue backing up"

- alert: LowCacheHitRate
  expr: rate(cache_hits_total[5m]) / (rate(cache_hits_total[5m]) +
rate(cache_misses_total[5m])) < 0.8
  for: 15m
  labels:
    severity: warning
  annotations:
    summary: "Cache hit rate below 80%"

- alert: HighErrorRate
  expr: rate(errors_total[5m]) > 10
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "Error rate exceeds 10/sec"
```

Deployment Plan

Docker Compose (Development/Staging)

```
# docker-compose.yml
version: '3.8'

services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://postgres:postgres@db:5432/nutricoach
      - REDIS_URL=redis://redis:6379
      - NODE_ENV=production
    depends_on:
      - db
      - redis
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/api/health"]
      interval: 30s
      timeout: 10s
      retries: 3

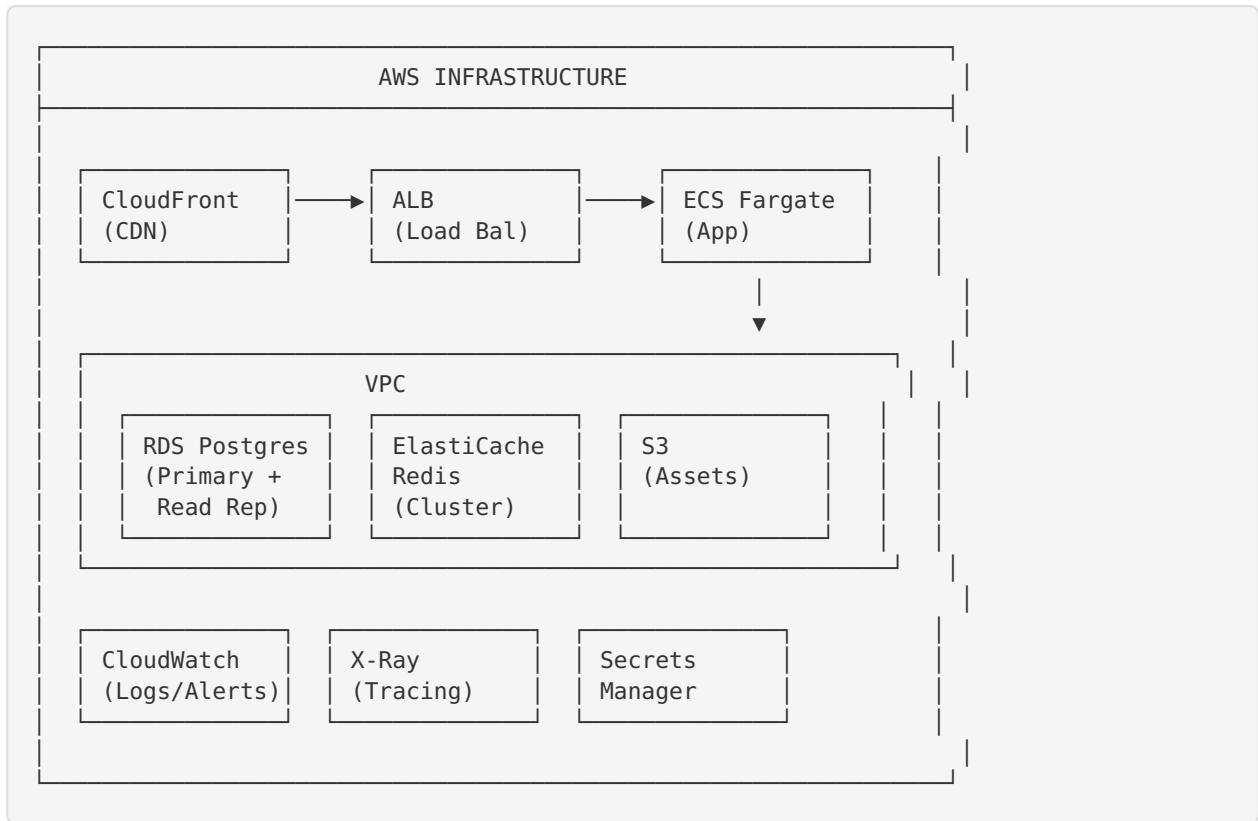
  worker:
    build: .
    command: node dist/workers/index.js
    environment:
      - DATABASE_URL=postgresql://postgres:postgres@db:5432/nutricoach
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis

  db:
    image: pgvector/pgvector:pg16
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=nutricoach
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
    ports:
      - "5432:5432"

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data
    ports:
      - "6379:6379"

volumes:
  postgres_data:
  redis_data:
```

Production Infrastructure (AWS)



Kubernetes (Alternative)

```
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nutricoach-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nutricoach
  template:
    metadata:
      labels:
        app: nutricoach
    spec:
      containers:
        - name: app
          image: nutricoach:latest
          ports:
            - containerPort: 3000
          resources:
            requests:
              memory: "512Mi"
              cpu: "250m"
            limits:
              memory: "1Gi"
              cpu: "500m"
          livenessProbe:
            httpGet:
              path: /api/health
              port: 3000
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /api/health
              port: 3000
            initialDelaySeconds: 5
            periodSeconds: 5
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nutricoach-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nutricoach-app
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Scaling Plan

Phase 1: MVP (0 - 10K users)

Infrastructure:

- Single Next.js instance on Abacus.AI hosting
- Managed PostgreSQL (single instance)
- No Redis (use in-memory caching)
- S3 for file storage

Optimizations:

- ISR for public pages
- Database connection pooling (PgBouncer)
- Basic logging

Estimated Costs: ~\$50-100/month

Phase 2: Growth (10K - 100K users)

Infrastructure Changes:

- Add Redis (ElastiCache) for caching + rate limiting
- PostgreSQL read replica for analytics queries
- Introduce job queue (BullMQ) for async processing
- CDN for static assets

Optimizations:

- Implement leaderboard snapshots
- Add table partitioning for events
- Cache food catalog in Redis
- Implement proper observability stack

Estimated Costs: ~\$300-500/month

Phase 3: Scale (100K - 500K users)

Infrastructure Changes:

- Multiple app instances behind load balancer
- Redis cluster mode
- PostgreSQL with multiple read replicas
- Dedicated worker nodes for job processing
- pgvector for RAG (separate from main DB if needed)

Optimizations:

- Implement connection pooling at scale
- Add circuit breakers for LLM calls
- Optimize database queries with better indexing
- Consider read-through caching patterns

Estimated Costs: ~\$1,500-2,500/month

Phase 4: Production Scale (500K - 1M users)

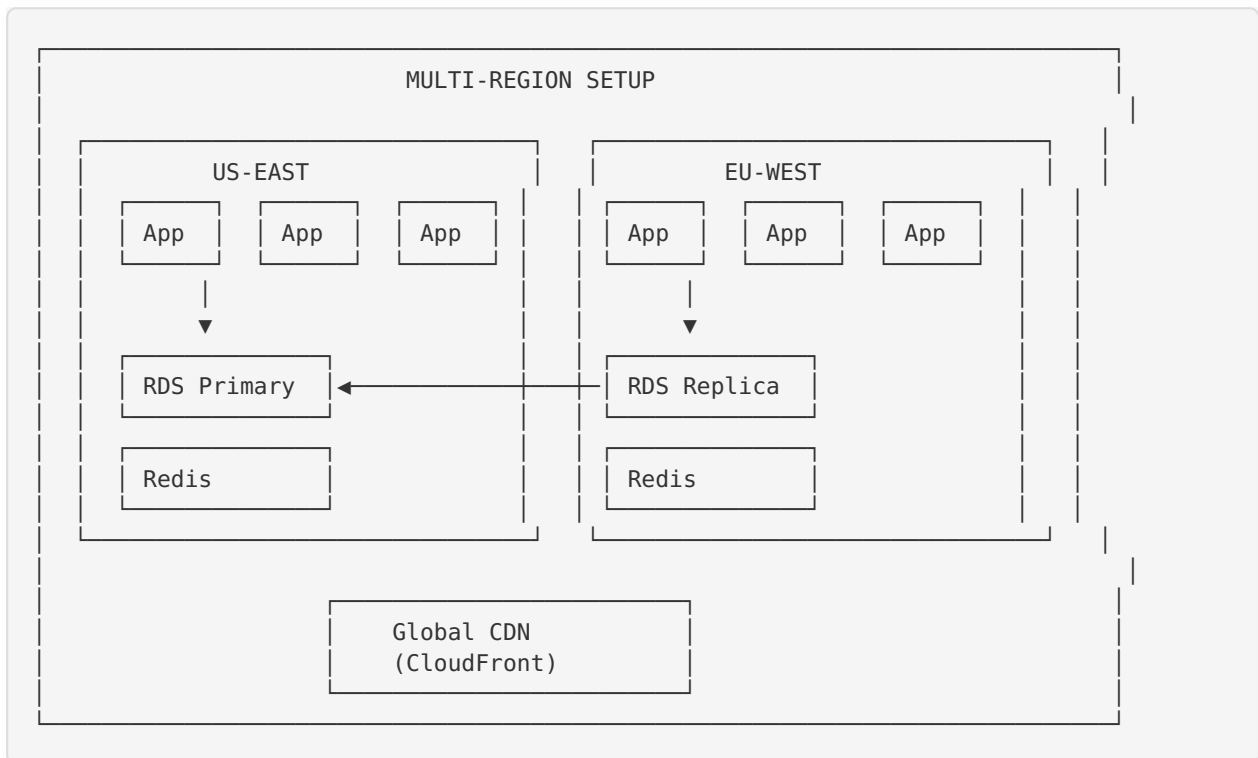
Infrastructure Changes:

- Multi-region deployment
- Global CDN with edge caching
- Database sharding consideration for specific tables
- Dedicated RAG infrastructure
- Event streaming (optional: Kafka/SQS for high-volume events)

Optimizations:

- Full CQRS for read-heavy operations
- Aggressive caching with smart invalidation
- Database query optimization and denormalization where needed
- Consider dedicated services for critical paths

Architecture Evolution:



Estimated Costs: ~\$5,000-10,000/month

Key Metrics to Monitor

Metric	Target (MVP)	Target (1M)
P50 Latency	< 200ms	< 100ms
P95 Latency	< 1s	< 500ms
P99 Latency	< 2s	< 1s
Cache Hit Rate	> 70%	> 90%
Error Rate	< 1%	< 0.1%
Queue Processing Time	< 30s	< 10s
Uptime	99.5%	99.9%
Database Connections	< 50	< 500
Redis Memory	< 1GB	< 10GB

Summary

This architecture provides:

1. **Modularity** - Services can be extracted to microservices as needed
2. **Scalability** - Horizontal scaling with clear bottleneck identification
3. **Performance** - Heavy caching, async processing, snapshot-based reads
4. **Observability** - Structured logging, tracing, metrics, and alerts
5. **Cost Efficiency** - Start small, scale as needed
6. **Maintainability** - Clear separation of concerns, versioned APIs

The key principle is **progressive enhancement**: start with a well-structured monolith and extract services only when specific scaling needs demand it.